College Of Engineering

Department Of Software Engineering

Selected Topics in Software Engineering: Assignment 1

Name	ID Number
Miheret Girmachew	ETS 1071/14
Mihret Desalegn	ETS 1074/14
Milkias Yeheyis	ETS 1101/14
Mikiyas Alemayehu Mekonen	ETS 1086/14
Mikiyas Alemayehu Gebrewold	ETS 1087/14
Nebat Hussen	ETS1278/14
Natnael Abayneh	ETS 1237/14

Submitted to: Inst. Gizate

Submission date: 03/04/2026

Abstract

Blockchain technology has rapidly transitioned from a niche concept underlying cryptocurrency systems to a broadly applicable paradigm within modern software engineering. This paper examines blockchain as an emerging topic in software engineering by exploring its foundational concepts, the new opportunities it introduces, and the significant challenges it poses to developers and researchers alike. The study draws on a review of recent academic literature to identify key trends, methodological shifts, and remaining research gaps. Findings indicate that blockchain enables decentralized application architectures, enhances data integrity, and fosters trust in multi-party systems through smart contracts and consensus mechanisms. However, critical concerns persist around scalability, energy efficiency, development complexity, and regulatory compliance. The document further surveys prominent development platforms such as Ethereum and Hyperledger Fabric, as well as supporting tools and programming languages. The discussion concludes that while blockchain holds considerable promise for transforming software engineering practices across industries including finance, healthcare, and supply chain management, realizing its full potential requires targeted advances in tooling, standardization, and cross-disciplinary collaboration.

1. Introduction 3
2. Overview of Blockchain Technology 5
2.1 Definition 5
2.2 Key Concepts 5
2.3 Types of Blockchain 5
2.4 Basic Architecture 5
3. New Opportunities in Software Engineering 6
3.1 Decentralized Applications (DApps) 6
3.2 Smart Contracts 6
3.3 Improved Security and Trust 6
3.4 Industry Use Cases 6
4. New Challenges in Software Engineering 7
4.1 Scalability 7
4.2 Performance Limitations 7
4.3 Energy Consumption 7
4.4 Development Complexity 7
4.5 Security Vulnerabilities 8
4.6 Regulatory and Legal Concerns 8
5. New Methods, Technologies, and Tools 8
5.1 Blockchain Platforms 8
5.2 Development Tools and Frameworks 9
5.3 Smart Contract Languages 9
5.4 Testing and Security Analysis Tools 9
5.5 Integration with Existing Systems 9
6. Literature Review 10
7. Conclusion 12
References 14

1. Introduction

Software engineering has undergone profound transformation since its formal recognition as a discipline in the late 1960s. From structured programming and object-oriented design to agile methodologies and cloud-native architectures, the field continuously adapts to technological innovation. An "emerging topic" in this context refers to a technology or paradigm that is gaining research traction, attracting practitioner interest, and showing demonstrable potential to reshape software design, development, or deployment practices, yet has not been fully absorbed into mainstream practice.

Blockchain technology satisfies these criteria convincingly. Originally conceived by Satoshi Nakamoto in 2008 as the underlying infrastructure for the Bitcoin cryptocurrency [1], blockchain has since evolved into a general-purpose distributed ledger technology with applications spanning financial services, healthcare, logistics, identity management, and government systems. At its core, a blockchain is a decentralized, tamper-resistant record-keeping system that operates through a network of participating nodes rather than a central authority.

The purpose of this paper is to provide a comprehensive academic examination of blockchain as an emerging topic within software engineering. The scope encompasses an overview of the technology's fundamental concepts, an assessment of the opportunities and challenges it introduces for software engineers, a review of relevant tools and platforms, a synthesis of the current academic literature, and a forward-looking discussion of trends and research directions. By the end of this paper, the reader should have a clear and well-rounded understanding of how blockchain intersects with, and meaningfully impacts, the practice of software engineering.

2. Overview of Blockchain Technology

2.1 Definition

A blockchain is a distributed digital ledger that records transactions across a network of computers in a manner that prevents retroactive alteration. Each entry, known as a block, contains a set of validated transactions, a cryptographic hash of the preceding block, a timestamp, and metadata. By chaining blocks together through their hashes, the structure forms an append-only history that is practically impossible to tamper with without invalidating the entire subsequent chain [2].

2.2 Key Concepts

Several foundational concepts distinguish blockchain from conventional database systems. Decentralization eliminates the single point of control by distributing data and processing authority across all participating nodes. Immutability ensures that once a block is confirmed, its contents cannot be changed without network consensus, providing a high degree of data integrity. Consensus mechanisms are the protocols by which nodes agree on the validity of transactions; prominent examples include Proof of Work (PoW), Proof of Stake (PoS), and Practical Byzantine Fault Tolerance (PBFT). The distributed ledger itself is replicated across all nodes, meaning no single entity holds exclusive control over the historical record [3].

2.3 Types of Blockchain

Blockchains are broadly categorized into three types. Public blockchains, such as Bitcoin and Ethereum, are permissionless networks open to any participant. Private blockchains restrict participation to authorized entities and are often used for internal enterprise applications. Consortium blockchains, such as those built on Hyperledger Fabric, are governed by a predefined group of organizations and offer a balance between openness and control. The choice among these types is a critical architectural decision for software engineers designing blockchain-based systems [4].

2.4 Basic Architecture

The architectural components of a blockchain system include nodes, which are individual participants maintaining a copy of the ledger;

blocks, which are data structures containing transaction records; and hashing functions, typically SHA-256, which link blocks and detect tampering. Nodes communicate via a peer-to-peer network protocol, propagating new transactions and blocks in real time. Smart contract platforms add an execution layer, allowing code to run deterministically on all nodes whenever specified conditions are met [5].

3. New Opportunities in Software Engineering

3.1 Decentralized Applications (DApps)

One of the most significant opportunities that blockchain affords software engineers is the ability to build decentralized applications, commonly referred to as DApps. Unlike traditional applications that rely on centralized servers, DApps operate on a peer-to-peer blockchain network, offering enhanced availability and censorship resistance. Developers must adopt new architectural patterns, such as separating business logic into smart contracts while delivering the user interface through conventional web or mobile frameworks [6].

3.2 Smart Contracts

Smart contracts are self-executing programs stored on a blockchain that automatically enforce the terms of an agreement when predefined conditions are met. Introduced systematically by the Ethereum platform, smart contracts reduce reliance on intermediaries, lower transaction costs, and increase the speed of multi-party processes. In supply chain management, for example, a smart contract can automatically release payment to a supplier upon verified delivery of goods, without requiring manual reconciliation [7].

3.3 Improved Security and Trust

Blockchain's cryptographic architecture provides a native mechanism for ensuring data authenticity and non-repudiation. Because all transactions are digitally signed and permanently recorded, it becomes straightforward to audit system activity and verify the origin of data. This quality is particularly valuable in industries handling sensitive information, such as healthcare, where patient records must be both accessible to authorized parties and resistant to unauthorized modification.

3.4 Industry Use Cases

The applicability of blockchain extends across multiple sectors. In finance, blockchain streamlines cross-border payments and enables decentralized finance (DeFi) protocols that operate without traditional banking infrastructure. In healthcare, blockchain is used to manage electronic health records and track pharmaceutical supply chains, reducing counterfeiting risks. In logistics, companies such as Walmart and Maersk have implemented blockchain-based tracking systems to increase supply chain transparency and reduce fraud [8].

4. New Challenges in Software Engineering

4.1 Scalability

A fundamental limitation of blockchain systems, particularly public ones, is their limited transaction throughput. Bitcoin processes approximately 7 transactions per second, while Ethereum historically managed around 15, figures that are orders of magnitude lower than those of traditional payment networks such as Visa, which handles thousands per second [3]. Addressing scalability without compromising decentralization or security, a challenge encapsulated in the "blockchain trilemma," remains an open research problem.

4.2 Performance Limitations

Beyond raw throughput, blockchain systems introduce latency due to block confirmation times and consensus delays. Distributed storage of the entire ledger across all nodes also incurs significant storage and network overhead. These performance characteristics make blockchain unsuitable for applications requiring near-real-time responsiveness, and software engineers must carefully evaluate whether the trade-offs are justified for a given use case.

4.3 Energy Consumption

Proof of Work consensus, used by Bitcoin, requires participants to expend substantial computational resources to validate blocks, resulting in energy consumption comparable to that of small nations. Although alternative consensus mechanisms such as Proof of Stake are significantly more energy-efficient, the environmental impact of blockchain infrastructure remains a concern for organizations with sustainability commitments [9].

4.4 Development Complexity

Building reliable blockchain applications demands expertise across cryptography, distributed systems, and specialized programming languages such as Solidity. The developer tooling ecosystem, while growing, remains less mature than that available for conventional web or enterprise application development. Debugging smart contracts is particularly challenging because deployed code is immutable; errors cannot simply be patched after release.

4.5 Security Vulnerabilities

Despite blockchain's inherent resistance to tampering, the smart contracts deployed on these networks are not immune to bugs. High-profile incidents, such as the 2016 DAO hack in which approximately $60 million worth of Ether was drained through a reentrancy vulnerability, underscore the critical importance of thorough smart contract auditing. Common vulnerability classes include reentrancy attacks, integer overflow and underflow, and improper access control [5].

4.6 Regulatory and Legal Concerns

Blockchain's decentralized and pseudonymous nature conflicts with existing regulatory frameworks in many jurisdictions. Data protection regulations such as the European Union's General Data Protection Regulation (GDPR) present a particular challenge, as the right to erasure is fundamentally incompatible with blockchain's immutability. Legal recognition of smart contracts, cross-border jurisdictional issues, and anti-money-laundering compliance further complicate the regulatory landscape for blockchain application developers.

5. New Methods, Technologies, and Tools

5.1 Blockchain Platforms

Ethereum is the most widely used smart contract platform, supporting a large developer community and a rich ecosystem of decentralized applications. Its transition from Proof of Work to Proof of Stake, completed in 2022, significantly reduced its energy consumption. Hyperledger Fabric, developed under the Linux Foundation, is a permissioned blockchain framework designed for enterprise applications, offering modular architecture and plug-in consensus algorithms. Other notable platforms include Solana, known for high throughput, and Polkadot, which enables interoperability between separate blockchain networks [6].

5.2 Development Tools and Frameworks

The Truffle Suite provides a comprehensive development environment for Ethereum DApps, including a testing framework, deployment pipeline, and interactive console. Hardhat is a newer, highly customizable development environment that has gained rapid adoption due to its plugin architecture and superior debugging capabilities. Remix IDE is a browser-based integrated development environment particularly suited to beginners, offering real-time compilation and a built-in debugger for Solidity contracts.

5.3 Smart Contract Languages

Solidity, a statically typed, object-oriented language with syntax inspired by JavaScript and C++, is the dominant language for writing smart contracts on Ethereum. Vyper offers a Python-like alternative with a deliberate emphasis on security and auditability, sacrificing some expressiveness to reduce the attack surface. For Hyperledger Fabric, chaincode can be written in Go, Java, or Node.js, reflecting the platform's enterprise-oriented flexibility.

5.4 Testing and Security Analysis Tools

Automated testing of smart contracts is supported by frameworks such as Mocha and Chai when used with Truffle, as well as Hardhat's native testing utilities. Security analysis tools such as MythX, Slither, and Manticore perform static and dynamic analysis to detect common vulnerability patterns before deployment. Formal verification tools, including the K framework applied to the Ethereum Virtual Machine, provide mathematical proofs of contract correctness for high-assurance applications [10].

5.5 Integration with Existing Systems

Integrating blockchain into legacy enterprise architectures presents both technical and organizational challenges. Oracle services, such as Chainlink, bridge the gap between on-chain smart contracts and off-chain real-world data sources. APIs and middleware layers allow conventional applications to interact with blockchain networks without requiring a full architectural overhaul. Enterprise integration patterns and event-driven architectures are commonly employed to synchronize blockchain state with traditional databases.

6. Literature Review

The academic literature on blockchain and software engineering has expanded substantially since 2016, reflecting growing practitioner and research interest. This section synthesizes key findings from a representative selection of influential studies, highlighting areas of consensus, divergence, and unresolved questions.

Zheng et al. [2] provide one of the earliest and most cited comprehensive surveys of blockchain technology, detailing its architecture, consensus mechanisms, and application scenarios. The authors conclude that while blockchain offers compelling advantages in terms of decentralization and security, scalability and privacy remain the most pressing research challenges. Their taxonomy of blockchain systems continues to serve as a foundational reference in subsequent literature.

Porru et al. [6] examine blockchain specifically from a software engineering perspective, arguing that the paradigm necessitates new development methodologies, testing strategies, and quality attributes. Their study highlights the absence of mature software engineering practices tailored to blockchain projects and calls for the community to develop specialized process models and metrics. This gap remains partially unaddressed in the literature as of the time of writing.

Luu et al. [5] conduct a systematic security analysis of Ethereum smart contracts, identifying and formalizing several classes of vulnerabilities including transaction order dependence, mishandled exceptions, and reentrancy. The paper introduces Oyente, one of the first automated symbolic execution tools for smart contract verification, and demonstrates its effectiveness in detecting known vulnerabilities in real-world contracts. Their work established a research direction that has since produced numerous subsequent security analysis tools.

Dinh et al. [4] provide a rigorous performance evaluation of four prominent blockchain platforms, BLOCKBENCH, Ethereum, Hyperledger Fabric, and Parity, through a unified benchmarking framework. Their findings reveal significant performance disparities depending on workload type and consensus mechanism, and underscore the importance of empirical evaluation in selecting a blockchain platform for a specific application context.

Crosby et al. [3] offer an accessible and widely referenced overview of blockchain applications across industries, providing early evidence of the technology's versatility beyond cryptocurrency. Their review of use cases in financial services, healthcare, and public records management helped frame early academic discourse on blockchain adoption. However, the rapid pace of development means that some of their specific platform assessments are now dated, illustrating the challenge of keeping literature reviews current in this fast-moving field.

Taken together, the literature reveals a clear trend toward greater specialization in blockchain research, with growing attention to specific application domains, security tooling, and performance optimization. A persistent gap is the limited availability of controlled empirical studies comparing blockchain-based and conventional solutions for equivalent problems, which makes rigorous cost-benefit analysis difficult for practitioners.

7. Conclusion

A critical assessment of blockchain in software engineering reveals a technology that is both genuinely transformative in specific contexts and frequently over-applied in contexts where it offers no meaningful advantage over conventional solutions. The decision to adopt blockchain should be driven by a rigorous evaluation of whether the problem genuinely requires decentralization, immutability, and trustless verification, or whether a traditional database with appropriate access controls would suffice.

The trade-offs inherent in blockchain adoption are substantial. On the benefit side, blockchain can eliminate the need for trusted intermediaries, reduce settlement times in financial transactions, provide an auditable and tamper-evident record, and enable novel business models through tokenization and decentralized governance. On the limitation side, it introduces significant complexity in development and operations, imposes performance constraints that disqualify it for many high-frequency transaction scenarios, and carries unresolved regulatory risk in most jurisdictions.

Looking forward, several research and development directions appear particularly promising. Layer 2 scaling solutions, such as Ethereum's rollup technologies, process transactions off-chain while inheriting the security guarantees of the underlying blockchain, potentially resolving the scalability trilemma. Cross-chain interoperability protocols aim to allow different blockchain networks to communicate and exchange value, addressing the current fragmentation of the ecosystem. Advances in zero-knowledge proof cryptography offer a path toward privacy-preserving blockchain applications that comply with data protection regulations without sacrificing auditability [11]. From a software engineering discipline perspective, the field would benefit from the development of standardized process models for blockchain application development, analogous to existing models for web or embedded systems development. The establishment of formal quality metrics, comprehensive testing methodologies, and best practice guidelines for smart contract development would substantially reduce the incidence of costly security incidents and accelerate the responsible adoption of the technology.

In summary, this paper has presented a structured academic examination of blockchain as an emerging topic in software engineering. Beginning with a grounding in the technology's fundamental architecture and concepts, the discussion progressed through the opportunities

blockchain creates — including decentralized application development, smart contract automation, and enhanced trust in multi-party systems — before addressing the substantial challenges it poses in terms of scalability, performance, security, and regulatory compliance. The survey of available platforms, tools, and languages reveals a maturing but still evolving ecosystem, while the literature review confirms that research interest is high and growing, with important gaps remaining in empirical comparative studies and domain-specific engineering methodologies. Ultimately, blockchain occupies a meaningful and expanding role in software engineering, not as a universal solution, but as a powerful paradigm for specific categories of application where decentralization, auditability, and censorship resistance are genuinely required. Software engineers who develop a deep understanding of both its capabilities and its limitations will be well positioned to deliver high-quality, appropriate blockchain solutions as the technology continues to mature.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," Cryptography Mailing List, Oct. 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An overview of blockchain technology: Architecture, consensus, and future trends," in Proc. IEEE 6th Int. Congr. Big Data, Honolulu, HI, USA, 2017, pp. 557–564.

[3] M. Crosby, P. Pattanayak, S. Verma, and V. Kalyanaraman, "Blockchain technology: Beyond bitcoin," Appl. Innov. Rev., vol. 2, pp. 6–19, 2016.

[4] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, "BLOCKBENCH: A framework for analyzing private blockchains," in Proc. ACM Int. Conf. Manage. Data (SIGMOD), Chicago, IL, USA, 2017, pp. 1085–1100.

[5] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Vienna, Austria, 2016, pp. 254–269.

[6] S. Porru, A. Pinna, M. Marchesi, and R. Tonelli, "Blockchain-oriented software engineering: Challenges and new directions," in Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. Companion (ICSE-C), Buenos Aires, Argentina, 2017, pp. 169–171.

[7] N. Szabo, "Formalizing and securing relationships on public networks," First Monday, vol. 2, no. 9, Sep. 1997.

[8] M. Iansiti and K. R. Lakhani, "The truth about blockchain," Harvard Bus. Rev., vol. 95, no. 1, pp. 118–127, Jan.–Feb. 2017.

[9] K. J. O'Dwyer and D. Malone, "Bitcoin mining and its energy footprint," in Proc. 25th IET Irish Signals Syst. Conf. 2014 China-Ireland Int. Conf. Inf. Commun. Technol. (ISSC 2014/CIICT 2014), Limerick, Ireland, 2014, pp. 280–285.

[10] E. Hildenbrandt et al., "KEVM: A complete formal semantics of the Ethereum virtual machine," in Proc. 31st IEEE Comput. Secur. Found. Symp. (CSF), Oxford, UK, 2018, pp. 204–217.

[11] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from bitcoin," in Proc. IEEE Symp. Secur. Privacy (SP), San Jose, CA, USA, 2014, pp. 459–474.

[12] A. M. Antonopoulos and G. Wood, Mastering Ethereum: Building Smart Contracts and DApps. Sebastopol, CA, USA: O'Reilly Media, 2018.